



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Aisyah Ramadani

NIM : 2428240037

Kelas : SI5B

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

Identitas Praktikum

Nama Mahasiswa	Aisyah Ramadani
NPM	2428240037
Kelas	SI5B
Pertemuan Ke-	5
Topik / Judul Praktikum	RESTful API dengan Express.js: Routing, Middleware, dan CORS
Tanggal Pelaksanaan	24 September 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Memahami konsep dan prinsip RESTful API serta penggunaan metode HTTP (GET, POST, PUT) untuk operasi data.
2. Mampu membangun server web menggunakan Express.js dengan beberapa route, termasuk route dinamis (:id) dan query string.
3. Mampu menerapkan middleware (logger, express.json, dan cors) serta menyimpan konfigurasi dalam file .env menggunakan dotenv.

II. Dasar Teori

RESTful API. REST (Representational State Transfer) adalah gaya arsitektur untuk membangun layanan web di mana setiap data diperlakukan sebagai resource yang diakses melalui URL. Operasi terhadap resource dinyatakan dengan metode HTTP: GET untuk membaca, POST untuk menambah, PUT untuk memperbarui, dan DELETE untuk menghapus. Server membalas dengan data (umumnya JSON) beserta kode status HTTP, misalnya 200 (berhasil), 201 (data dibuat), 400 (permintaan tidak valid), dan 404 (data tidak ditemukan).

Node.js dan Express.js. Node.js adalah lingkungan runtime JavaScript di sisi server. Express.js adalah framework web minimalis di atas Node.js yang menyediakan routing dan middleware sehingga pembuatan API menjadi lebih ringkas.

Routing. Routing menentukan bagaimana aplikasi merespons permintaan ke suatu URL dan metode HTTP tertentu. Data dari klien dapat dibaca melalui parameter route

(req.params, contoh /mahasiswa/:id), query string (req.query, contoh ?jurusan=...), dan isi permintaan (req.body).

Middleware. Middleware adalah fungsi yang berjalan di antara permintaan dan respons dan memiliki akses ke req, res, serta next(). Pada praktikum ini digunakan middleware buatan sendiri (logger untuk mencatat setiap request), express.json() agar body JSON dapat dibaca, dan cors untuk mengatur asal (origin) yang boleh mengakses API.

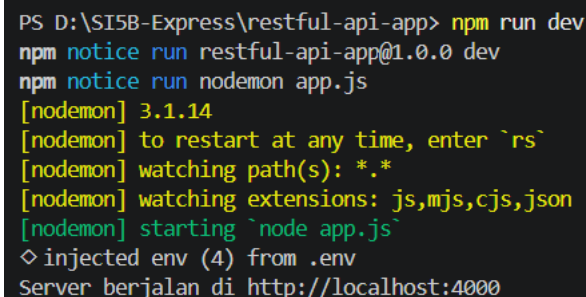
III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Runtime JavaScript sisi server (versi sesuai hasil perintah node -v)
2	Visual Studio Code	Editor kode
3	Express.js	Versi 5.2.1 (framework web)
4	cors, dotenv, nodemon	cors 2.8.6, dotenv 18.0.4, nodemon 3.1.14 (paket npm)
5	Peramban web	Menguji endpoint dan halaman Frontend- uji/index.html
6	Git dan GitHub	Version control dan penyimpanan repository

IV. Langkah Kerja

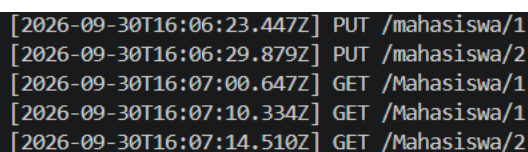
1. Membuat folder proyek restful-api-app, lalu menjalankan npm init untuk membuat package.json.
2. Memasang dependensi dengan perintah npm install express cors dotenv, serta npm install --save-dev nodemon. Pada package.json ditambahkan script "start": "node app.js" dan "dev": "nodemon app.js".
3. Membuat file .env (berisi PORT dan CORS_ORIGIN) dan .env.example sebagai contoh, lalu menulis .gitignore yang mengabaikan node_modules/ dan .env.
4. Membuat app.js: memuat dotenv, membuat aplikasi Express, menulis middleware logger yang mencatat waktu, metode, dan URL setiap request, kemudian mendaftarkan logger, cors (origin dari CORS_ORIGIN, metode GET/POST/PUT/DELETE), dan express.json().
5. Membuat route dasar: GET / (mengirim teks "Server Express.js berjalan!") dan GET /profile, kemudian menjalankan server dengan app.listen pada PORT dari .env.
6. Membuat data sementara berupa array mahasiswa di memori dan endpoint CRUD: GET /mahasiswa (dengan filter ?jurusan=), GET /mahasiswa/:id, POST /mahasiswa (validasi nama dan jurusan, status 201), dan PUT /mahasiswa/:id (status 404 jika id tidak ada).
7. Menjalankan server dengan npm run dev, lalu menguji setiap endpoint melalui peramban atau aplikasi pengujian API dan memeriksa log yang muncul di terminal.
8. Mengunggah proyek ke repository GitHub.

V. Hasil dan Pembahasan



```
PS D:\SI5B-Express\restful-api-app> npm run dev
npm notice run restful-api-app@1.0.0 dev
npm notice run nodemon app.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
◇ injected env (4) from .env
Server berjalan di http://localhost:4000
```

Gambar 1. Server berhasil dijalankan dengan nodemon pada port 4000.



```
[2026-09-30T16:06:23.447Z] PUT /mahasiswa/1
[2026-09-30T16:06:29.879Z] PUT /mahasiswa/2
[2026-09-30T16:07:00.647Z] GET /Mahasiswa/1
[2026-09-30T16:07:10.334Z] GET /Mahasiswa/1
[2026-09-30T16:07:14.510Z] GET /Mahasiswa/2
```

Gambar 2. Log request GET dan PUT pada endpoint /mahasiswa.

Pembahasan:

Gambar 1 Server dijalankan dengan perintah `npm run dev`, yang menjalankan `nodemon` `app.js` sesuai script pada `package.json`. `Nodemon` memantau perubahan file berekstensi `js`, `mjs`, `cjs`, dan `json`, sehingga server restart otomatis setiap kode disimpan dan pengembangan tidak perlu menjalankan ulang server secara manual. Pesan injected env (4) from `.env` menunjukkan `dotenv` berhasil memuat empat variabel lingkungan, termasuk `PORT`. Server aktif di `http://localhost:4000`, sesuai nilai `PORT` pada file `.env`, sehingga semua pengujian endpoint memakai alamat tersebut.

Gambar 2 Log pada terminal dihasilkan oleh middleware logger yang mencatat waktu, metode, dan URL setiap request yang masuk. Request `GET /` mengakses route dasar, `GET /mahasiswa` mengambil seluruh data, dan `GET /mahasiswa/1` serta `/mahasiswa/2` mengambil satu data melalui route dinamis `:id`. Request `PUT /mahasiswa/1` dan `/mahasiswa/2` memperbarui data mahasiswa, lalu `GET /mahasiswa/1` dan `/mahasiswa/2` dipanggil lagi untuk memastikan perubahan tersimpan. Terdapat pula `PUT /mahasiswa` tanpa id yang tidak sesuai dengan route `PUT /mahasiswa/:id`, sehingga tidak dilayani route mana pun dan menghasilkan status 404. Penulisan `/Mahasiswa` dengan huruf kapital tetap dikenali karena Express secara bawaan tidak membedakan huruf besar dan kecil pada path route. Semua request tercatat karena logger didaftarkan sebelum route dan memanggil `next()` untuk meneruskan request ke handler berikutnya.

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	Data hilang setiap server dijalankan ulang.	Data disimpan di array memori sehingga bersifat sementara; untuk data permanen perlu memakai basis data.

VII. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. RESTful API sederhana berhasil dibuat dengan Express.js menggunakan metode `GET`, `POST`, dan `PUT` untuk mengelola data mahasiswa, lengkap dengan kode status HTTP yang sesuai (200, 201, 400, 404). Hal ini memenuhi tujuan pertama dan kedua.
2. Route dinamis (`:id`) dan query string (`?jurusan=`) berhasil diterapkan untuk mengambil dan menyaring data (tujuan kedua).

3. Middleware logger, express.json, dan cors berhasil diterapkan, dan konfigurasi PORT serta CORS_ORIGIN dipisahkan ke file .env dengan dotenv sehingga kode lebih aman dan fleksibel (tujuan ketiga).

Daftar Pustaka

Express. (2026). Express - Node.js web application framework. <https://expressjs.com/>

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine].

OpenJS Foundation. (2026). Node.js documentation. <https://nodejs.org/docs/latest/api/>

Lampiran

Tautan Repository GitHub:

https://github.com/aisyahramadani2428240037-gif/si5b_latihan_restful_AisyahRamadani

Tautan Deployment (jika ada):

<https://si5b-latihan-restful-nama.vercel.app>